

The Import Graph Answers a Different Question

Structure, Issue Text, and What Counts as a File You Needed to Read

NEIL LUNAVAT, [TODO: institution], [TODO: country]

To change an unfamiliar file, a developer or an agent must decide what else to read, and in what order. Two signals are available: the repository’s dependency structure, and the words of the issue report. Tools overwhelmingly use the first. We compare both, and their combination, on 2,412 change tasks drawn from 1,873 real pull requests in 113 Python repositories, scoring each ordering by how much of an evidence-based answer key it covers per line read.

The comparison only has an answer once one says what “needed to read” means, and that turns out to decide everything. Graded against the files that were *co-edited* in the same pull request, the four best orderings fuse structure with issue text, the best pure signal is issue-text-to-path matching (AUC 0.436), and the best purely structural ordering is tenth (0.389); a query-independent PageRank prior is seventeenth, barely above random (0.272 vs. 0.247). Graded against the files that *define symbols the change uses*, the ranking inverts: cost-scaled personalised PageRank is first (0.489), issue-text matching falls to thirteenth (0.383), and the PageRank prior climbs to tenth, far above random (0.410 vs. 0.251). Across the sixteen contending methods the two evidence sources induce rankings that are statistically unrelated (Spearman $\rho = 0.26$, $p = 0.34$). Every reversal is significant under Holm correction with the pull request as the unit.

Two further results explain the reversal and what to do about it. The two-hop neighbourhood of the seed, the usual justification for graph retrieval, holds 87.5% of the needed files but is also 56% of the repository and a median of 21,646 lines: it supplies recall, not precision, with a lift of only 1.57 $\times$ over taking the same share of files at random. And the two signals genuinely disagree about which files to name – the top-10 lists of personalised PageRank and issue-text matching have a mean Jaccard overlap of 0.19, and at $k = 20$ nearly a quarter of co-edited files are reachable by words alone. Reciprocal-rank fusion of one structural and one lexical ordering is consequently the only family in the top six under both evidence sources: it gains 0.069 AUC over pure structure on co-edited files ($p < 0.001$) while losing nothing on symbol files (-0.0004 , $p = 1.0$). We argue that retrieval for code change should be reported per evidence source, and that a cheap lexical signal belongs in every graph-based context tool.

Additional Key Words and Phrases: code search, context retrieval, dependency graphs, software maintenance, empirical software engineering

ACM Reference Format:

Neil Lunavat. 2026. The Import Graph Answers a Different Question: Structure, Issue Text, and What Counts as a File You Needed to Read. *ACM Trans. Softw. Eng. Methodol.* 1, 1 (September 2026), 10 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

A developer is handed a bug report and a file to change. Before writing anything they must decide what else to read. So must an automated agent, under a much harder constraint: a context window is finite, and every line spent on an irrelevant file is a line not spent on a relevant one.

The dominant answer in tool building is the dependency graph. Aider builds a repository map from the call and definition graph; RepoGraph, LocAgent and CoSIL construct code graphs and walk them outward from a starting point. The premise is intuitive and rarely stated: what you need to read is near, in the graph, to what you are changing.

There is a second signal, and it is the one a human reaches for first. Asked to fix the login page, a developer thinks of the authentication files – not because `login.py` imports them, which it may

Author’s address: Neil Lunavat, [TODO: institution], [TODO: country], neillunavat3192@gmail.com.

2026. 1049-331X/2026/9-ART \$15.00
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

well not, but because the words match. That signal sits in the issue report, and graph-based tools largely do not use it for ordering.

This paper compares the two. The comparison sounds simple and is not, because it has no answer until one says what it means for a file to have been “needed”. We use two evidence sources that can be recovered from the repository without human labelling: the files *co-edited* in the same pull request, and the files that *define symbols* the changed code references. Both are defensible. Neither is obviously the right one. They give opposite answers to the question this paper asks.

Research questions

RQ1 How much of a repository is the seed’s two-hop neighbourhood, and how much recall does it buy over a same-sized random sample?

RQ2 Do structural and lexical reading orders rank differently under the two evidence sources, and by how much?

RQ3 How much does conditioning on the seed file add over a query-independent repository prior, and does the answer depend on the evidence source?

RQ4 Do the two signals retrieve the same files, or different ones?

RQ5 Does fusing them produce an ordering that is good under both evidence sources?

Contributions

- (1) The first head-to-head comparison of import-graph proximity and issue-text retrieval as *reading orders* under a line budget, on 2,412 tasks from 1,873 real pull requests in 113 Python repositories.
- (2) Evidence that the conclusion of such a comparison is decided by the evidence source rather than by the methods: sixteen methods, two sources, rankings unrelated at $\rho = 0.26$.
- (3) A quantification of what graph proximity actually contributes: recall over half the repository, at a lift of 1.57 $\times$, with 4.2% precision at two hops.
- (4) A robustness criterion — worst-case rank across evidence sources — and the finding that under it every fusion of a structural and a lexical ordering beats every pure signal.
- (5) A replication package with the graphs, tasks, issue-token bags, orderings and analysis scripts.

2 BACKGROUND AND RELATED WORK

Dependency structure as a guide to change. Using program structure to decide what a change touches is old. Impact analysis [2] propagates from a changed element along dependencies; Robillard [11] ranks program elements by the topology of their dependencies; feature-location surveys [5] catalogue structural, textual and dynamic techniques and note that they are usually evaluated in isolation. Ko et al. [8] observed developers navigating largely by identifier names and search rather than by following dependencies.

Change history. Zimmermann et al. [15] and Ying et al. [13] mine co-change from version histories to suggest files that change together. Our co-edited evidence source is the same notion of relatedness, used as ground truth rather than as a method.

Graph-based context for agents. SWE-bench [7] and its successors [1, 14] made repository-scale change tasks a standard benchmark, and a family of systems now assembles context by walking a code graph: Aider’s repository map [6], RepoGraph [9], LocAgent [3] and CoSIL [4]. Agentless [12] is a notable counterexample that leans on file and directory names. What none of these report is how their conclusions would change under a different definition of which files a task required.

[TODO: survey 20–30 localization and agent papers and tabulate the ground truth each one uses; this table is the direct motivation for the paper and is not yet built]

Retrieval. BM25 [10] remains the reference lexical ranking function; we use it unchanged, varying only the query.

3 STUDY DESIGN

The corpus, the graph builder, the task definition and the coverage–cost metric were fixed and tagged in version control before any method was run; the present study adds method families and analyses on top of that frozen base.

3.1 Repositories and tasks

We draw pull requests from SWE-bench, SWE-bench-Live, SWE-Gym, SWE-bench Pro, Loc-Bench and SWE-rebench, keeping Python repositories with a permissive licence. A pull request that edits k existing non-test source files that are nodes of the import graph yields k tasks: each edited file in turn becomes the *seed*, and the task is to order the remaining files of the repository. This gives 3,123 tasks from 1,873 pull requests in 113 repositories. 2,412 tasks have at least one key file inside the graph once the seed is removed and are the ones scored here; 1,958 of them (708 pull requests) have a co-edited key and 2,341 (1,156 pull requests) a symbol key.

The median repository contributes 131 graph nodes and the median key is 3 files totalling 1,515 lines, so the task is to find a handful of files among roughly a hundred.

3.2 Import graphs

Graphs are built per base commit with a custom ast walker over the repository at that commit. Nodes are source files; an edge $a \rightarrow b$ means a imports a module resolving to b . Package `__init__` files are not inserted as intermediaries, from `X import Y` resolves to the module, edges are unweighted, imports inside `TYPE_CHECKING` blocks and inside function bodies are kept, and test files are excluded. Per-file line counts come from the same pass and are the cost unit throughout.

3.3 Two evidence sources

co_edited The other non-test source files changed by the same pull request. This is what a developer in fact touched, and it is independent of the import graph.

symbol Files that define names the changed code references, resolved with a static analyser. This is what the change depends on.

The seed file is removed from both. We deliberately do not merge them: the whole question is whether the choice matters. It does, and the union is not a neutral compromise — the symbol key contributes 7,675 file references against 4,458 for co-editing, so a union largely inherits the structural source’s verdict.

A third source — files actually executed by the tests that the fix repairs — is defined in our protocol but collected for only a small part of the corpus, and is therefore excluded here. [TODO: complete the execution-based key over the remaining pull requests and repeat the analysis with three sources]

3.4 Reading orders

Every method receives the graph at the base commit, the seed file, per-file identifier bags and the issue text, and returns an ordering of all other nodes. Table 1 lists them by family. Identifier bags are built by splitting every identifier in the file on underscores and camel-case boundaries and lower-casing; the issue text is tokenised identically, so text is matched against code over one

Table 1. Reading orders, by family. Structural methods use the seed and the graph; global methods use neither the seed nor the issue; lexical methods use no graph.

Family	Method	Ordering
structure	bfs_und	breadth-first from the seed, undirected
	hops_lines	by (hop distance from seed, file length)
	ppr_und	personalised PageRank, restart at the seed
	ppr_und_pl	the same, divided by file length
global	pagerank	PageRank of the whole graph; ignores seed and issue
	indegree	number of files importing this one
lexical	bm25_seed	BM25, query = identifiers of the seed file
	path_seed	BM25 over paths, query = the seed's path tokens
	bm25_issue	BM25, query = identifiers of the issue text
	bm25_issue_pl	the same, divided by file length
	path_issue	BM25 over file paths, query = issue tokens
fusion	rrf_hops_path	RRF of hops_lines and path_issue
	rrf_hops_issue_path	RRF of hops_lines, bm25_issue, path_issue
	rrf_pprpl_issue_path	RRF of ppr_und_pl, bm25_issue, path_issue
	rrf_ppr_issue	RRF of ppr_und and bm25_issue
	rrf_ppr_issue_path	RRF of ppr_und, bm25_issue, path_issue
reference	same_dir	files in the seed's directory first
	random	uniformly shuffled: the floor
	oracle	key files only, cheapest first: the ceiling

vocabulary. BM25 is used with the standard $k_1 = 1.5$, $b = 0.75$ and differs between methods only in the query. Fusion is reciprocal-rank fusion with $k = 60$.

Two points of fairness are worth stating. The structural methods are *given* a correct answer for free — the seed is one of the files the pull request changed. The issue-text methods are given no seed at all. And the metric charges by lines read, so we include cost-scaled variants (suffix _pl, score divided by file length) to separate “ranks the right files highly” from “happens to prefer short files”.

3.5 Metric

An ordering is read from the top until a line budget is exhausted; coverage is the fraction of the key that has been read. We evaluate at budgets {250, 500, 1000, 2000, 4000, 8000, 16000, 32000} and report the AUC, the mean coverage over those eight doublings, so that a method must be good at both tight and generous budgets. We also report the more directly interpretable quantity: the number of lines that must be read before half the key has been covered.

Orderings with ties are re-drawn three times per task with distinct seeds and the coverage curves averaged.

3.6 Statistics

The unit of analysis is the pull request: task AUCs are averaged within a pull request first, so a change touching many files does not count many times. Paired comparisons use the Wilcoxon signed-rank test with the Vargha–Delaney A_{12} effect size, Holm-corrected within each family of comparisons. Confidence intervals are 2,000-resample bootstraps resampling *repositories*, not tasks, so that a few large repositories cannot manufacture significance.

Table 2. Where the key files sit relative to the seed, and how big the two-hop ball is (medians over 2,412 tasks).

Key files at hop distance	1	2	3+	unreachable
co_edited	39.0%	49.1%	9.1%	2.8%
symbol	45.3%	41.4%	9.1%	4.2%
<i>The two-hop ball: 68 files, 21,646 lines, 56% of the repository</i>				
recall 87.5%, precision 4.2%, recall lift over a same-sized random sample 1.57×				
<i>The key: 3 files, 1,515 lines</i>				

4 RESULTS

4.1 RQ1: the two-hop neighbourhood is half the repository

“The files you need are within two hops” is true here: 86.7% of symbol-key files and 88.1% of co-edited files lie within two undirected hops of the seed (Table 2). Taken alone this reads as strong evidence for graph retrieval.

The other half of the statistic undoes it. In the median task the two-hop ball holds 68 of the repository’s 131 files and 21,646 lines — 56% of the files on average, and more than five times a 4,000-line budget. Its precision is 4.2%. Against a random sample of the same size it lifts recall by a factor of 1.57.

The two-hop neighbourhood is therefore not a filter. It is a statement that the files you need are somewhere in about half the repository — which is exactly the regime in which the *ordering* decides everything and the graph has already said all it has to say.

4.2 RQ2: the evidence source decides the winner

Table 3 is the central result. Under the co-edited key the top five orderings are four fusions and one lexical method; the best purely structural ordering is tenth and the query-independent priors are seventeenth and eighteenth, below *same_dir*. Under the symbol key the table inverts: cost-scaled personalised PageRank is first, three of the top four are structural, and the best lexical ordering is thirteenth.

The individual reversals are large and all significant after Holm correction. Issue-text path matching beats personalised PageRank by 0.078 AUC on co-edited files ($A_{12} = 0.597$, $p < 0.001$) and loses to it by 0.062 on symbol files ($A_{12} = 0.417$, $p < 0.001$). BM25 on the issue text beats it by 0.050 and then loses by 0.105 ($A_{12} = 0.364$).

Across the sixteen contending methods the two sources induce rankings that are not significantly related (Spearman $\rho = 0.26$, $p = 0.34$; including the three references, which sit at the same end of both rankings by construction, raises this only to $\rho = 0.45$, $p = 0.06$).

Figure 1 shows the same reversal as coverage curves. Table 4 states it in the unit a context budget is actually spent in: to reach half the co-edited files a reader following personalised PageRank consumes a median of 6,514 lines, against 4,747 following issue-text path matching and 3,254 following the best fusion. On symbol files the same three numbers are 4,395, 6,524 and 3,849.

4.3 RQ3: what the seed is worth depends on the source too

A query-independent PageRank prior — a ranking that never looks at the seed or the issue, and simply says which files matter in this repository — reaches 0.410 on symbol files against 0.446 for personalised PageRank, which does look at the seed. That is 92% of the personalised score, and

Table 3. Mean AUC by evidence source, pull request as the unit, with bootstrap 95% confidence intervals over repositories. The same nineteen orderings, ranked twice.

co_edited (708 PRs)				symbol (1,156 PRs)			
#	Method	Fam.	AUC [CI]	#	Method	Fam.	AUC [CI]
1	oracle	ref	0.773 [.750,.796]	1	oracle	ref	0.793 [.772,.813]
2	rrf_pprpl_issue_path	fus	0.488 [.457,.520]	2	ppr_und_pl	str	0.489 [.460,.517]
3	rrf_hops_issue_path	fus	0.482 [.450,.513]	3	rrf_hops_path	fus	0.477 [.450,.504]
4	rrf_hops_path	fus	0.458 [.426,.489]	4	hops_lines	str	0.477 [.447,.505]
5	rrf_ppr_issue_path	fus	0.446 [.415,.477]	5	rrf_pprpl_issue_path	fus	0.475 [.447,.503]
6	path_issue	lex	0.436 [.406,.466]	6	rrf_hops_issue_path	fus	0.458 [.430,.486]
7	bm25_issue	lex	0.407 [.379,.435]	7	ppr_und	str	0.446 [.417,.475]
8	bm25_seed	lex	0.393 [.362,.425]	8	rrf_ppr_issue_path	fus	0.421 [.392,.450]
9	rrf_ppr_issue	fus	0.393 [.363,.425]	9	bfs_und	str	0.420 [.391,.447]
10	hops_lines	str	0.389 [.354,.423]	10	pagerank	glo	0.410 [.381,.439]
11	ppr_und_pl	str	0.371 [.338,.403]	11	indegree	glo	0.398 [.371,.425]
12	path_seed	lex	0.368 [.335,.401]	12	rrf_ppr_issue	fus	0.391 [.361,.421]
13	bfs_und	str	0.362 [.330,.395]	13	path_issue	lex	0.383 [.352,.414]
14	ppr_und	str	0.357 [.325,.391]	14	path_seed	lex	0.365 [.335,.393]
15	bm25_issue_pl	lex	0.354 [.318,.389]	15	bm25_issue_pl	lex	0.355 [.324,.386]
16	same_dir	ref	0.327 [.296,.359]	16	bm25_issue	lex	0.340 [.311,.368]
17	pagerank	glo	0.272 [.240,.302]	17	bm25_seed	lex	0.336 [.305,.365]
18	indegree	glo	0.268 [.238,.299]	18	same_dir	ref	0.319 [.290,.348]
19	random	ref	0.247 [.218,.276]	19	random	ref	0.251 [.223,.278]

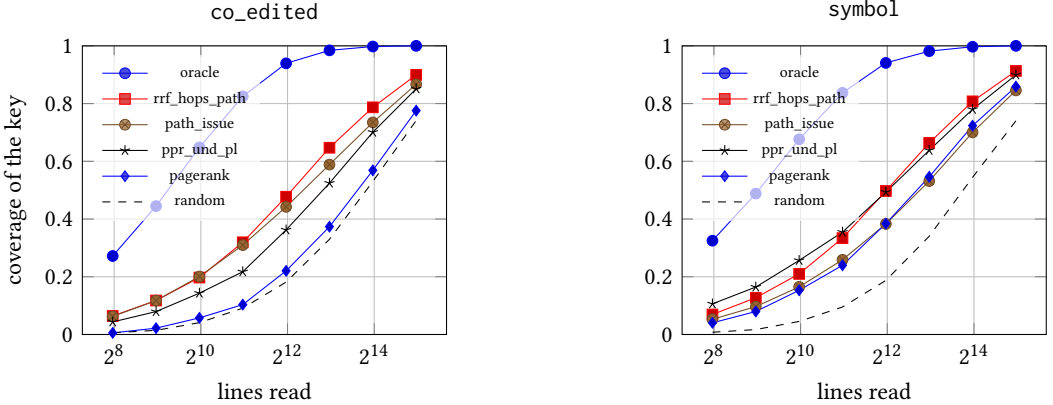


Fig. 1. Mean coverage against lines read. The same five orderings change places between the two evidence sources: issue-text path matching tracks the best fusion on co-edited files and falls below the query-independent prior on symbol files, while cost-scaled personalised PageRank does the reverse.

far above random (0.251; $A_{12} = 0.711$, $p < 0.001$). On that evidence source, most of what the personalised walk delivers is a repository prior.

Under the co-edited key the same prior scores 0.272 against a random floor of 0.247. The difference is significant but negligible ($A_{12} = 0.530$, $p = 0.004$), and personalised PageRank beats it by 0.086 ($A_{12} = 0.618$). Knowing which file is being changed therefore matters a great deal for predicting what else was changed, and rather little for predicting what the code calls, where knowing the repository's hubs is nearly as good.

Table 4. Median lines that must be read before half the key is covered. Lower is better.

Method	co_edited	symbol
oracle	524	440
rrf_pprpl_issue_path	3,254	3,849
rrf_hops_path	3,956	3,699
ppr_und_pl	6,155	3,483
hops_lines	5,620	4,080
ppr_und	6,514	4,395
path_issue	4,747	6,524
bm25_issue	5,012	7,451
pagerank	11,294	5,635
random	14,627	14,223

Table 5. How each key file is reached within the top k of the structural family (ppr_und, hops_lines) and the lexical family (bm25_issue, path_issue).

Source	k	both	structure only	words only	neither
co_edited	10	32.2%	11.4%	30.0%	26.4%
co_edited	20	50.7%	9.6%	24.4%	15.3%
co_edited	50	76.7%	5.9%	13.4%	4.0%
symbol	10	28.5%	23.3%	20.4%	27.8%
symbol	20	47.4%	19.9%	16.5%	16.2%
symbol	50	72.9%	11.5%	9.8%	5.9%

This is worth stating plainly, because the opposite claim is easy to make from a single source: whether “the graph knows what you are working on” appears true or false is itself a function of the answer key.

4.4 RQ4: the two signals name different files

The reversal is not two methods disagreeing about the order of one list. At $k = 10$ the top-10 sets of personalised PageRank and issue-text BM25 have a mean Jaccard overlap of 0.187, and of personalised PageRank and issue-text path matching only 0.102; at $k = 50$ these rise to 0.508 and 0.412. For comparison, personalised PageRank and the query-independent prior overlap at 0.364 at $k = 10$: a seed-conditioned walk resembles the repository prior more than it resembles a lexical ranking.

Table 5 follows individual key files. At $k = 20$, 24.4% of co-edited files are found only by the lexical family and 9.6% only by the structural family; on symbol files the asymmetry flips (19.9% structure-only against 16.5% words-only). Under either source about a sixth of key files are found by neither family at $k = 20$, which is where the remaining headroom to the oracle lives.

4.5 RQ5: fusion is the only ordering that survives both sources

If the evidence source is not known in advance — and for a tool shipped to users it is not — the quantity to optimise is the worst rank a method attains across sources. Table 6 ranks by it. Every one of the five fusions beats every pure signal. rrf_hops_path, which is nothing more than reciprocal-rank fusion of hop-distance ordering with issue-tokens-against-file-paths, is fourth under one source and third under the other.

Table 6. Worst rank attained across the two evidence sources. Lower is better. Every fusion beats every pure signal.

Method	co_edited	symbol	worst
rrf_hops_path	4	3	4
rrf_pprpl_issue_path	2	5	5
rrf_hops_issue_path	3	6	6
rrf_ppr_issue_path	5	8	8
rrf_ppr_issue	9	12	12
hops_lines	10	4	10
ppr_und_pl	11	2	11
bfs_und	13	9	13
path_issue	6	13	13
ppr_und	14	7	14
path_seed	12	14	14
bm25_issue_pl	15	15	15
bm25_issue	7	16	16
pagerank	17	10	17
bm25_seed	8	17	17
indegree	18	11	18

The insurance is close to free. Against pure structure (hops_lines) it gains 0.069 AUC on co-edited files ($A_{12} = 0.586$, $p < 0.001$) and loses 0.0004 on symbol files ($p = 1.0$); against ppr_und_pl it gains 0.087 ($p < 0.001$) and loses 0.012 ($p = 0.47$). The lexical signal adds a large amount where structure is weak and costs nothing where structure is strong.

4.6 Sensitivity: are the lexical methods just copying paths out of the issue?

Issue reports often contain stack traces, and a stack trace names files. If the lexical advantage came from reports quoting the paths of the files that had to change, it would be an artefact of the benchmark rather than a property of language.

It does not. The full path of a key file appears verbatim in the issue text for 4.2% of key files, and 12.6% of tasks contain at least one; the file stem appears for 20.4%. Repeating the analysis on the 87.4% of tasks whose issue quotes no key path leaves the ordering essentially unchanged: path_issue falls from 0.436 to 0.409 on co-edited files but keeps its position ahead of every structural method, and the fusions keep the top four places. The comparable figures for the seed itself are 10.3% full path and 35.1% stem.

The path_seed control separates “issue text” from “path matching” more directly: the same path-BM25 ranking, queried by the seed’s own path instead of the issue, scores 0.368 on co-edited files against 0.436 for the issue query (+0.067, $p < 0.001$), and the two are indistinguishable on symbol files (+0.018, $p = 0.13$). The gain is attributable to the issue text, and only where the target is what developers changed.

5 DISCUSSION

The graph and the issue answer different questions. Read together, the results have a simple interpretation. An import edge records that one file calls another. That is close to the definition of the symbol key, and structural methods duly win there. The co-edited key records what a person decided to change in one sitting — a decision made about a feature, a bug, a concept — and words

track that better than call structure does. Neither result is a defect in the other method. They are answers to different questions that the literature has been treating as one question.

The practical consequence is not “use words instead”. It is that a tool whose ordering rests on one signal is being tuned against whichever evidence source its evaluation happened to adopt. The fusion results show the cost of insurance is near zero: adding a BM25 ranking of issue tokens against file paths to an existing graph walk needs no model, no index beyond the file list and no extra hop through the graph, and it is the difference between being tenth and being fourth when the target turns out to be co-editing.

What “within two hops” is worth. The two-hop neighbourhood earns its reputation on recall and loses it on precision: it contains seven of every eight needed files and also half the repository. Reporting the recall without the size is, on this corpus, reporting half a statistic.

Recommendations. (1) Report retrieval results separately for at least one structural and one behavioural notion of relevance, and report the worst case across them. (2) Treat a query-independent repository prior, not random order, as the floor for structural methods — on symbol-style keys it captures 92% of a personalised walk. (3) Include a lexical signal from the issue text in graph-based context tools.

6 THREATS TO VALIDITY

Construct. Neither evidence source is ground truth about what a developer needed to read; they are recoverable proxies with opposite biases, which is why we report both and never merge them. Coverage per line assumes whole files are read, which overstates the cost for agents that read selectively. The execution-based source is absent (§3.3) and would be the natural third leg.

Internal. The graph builder resolves imports statically and misses dynamic imports and plugin registration. The symbol key is produced by a static analyser and inherits its resolution failures. Both biases favour structural methods, so the co-edited reversal is if anything understated. Issue-path leakage is measured and shown not to drive the result.

External. Python only, file granularity only, and pull requests drawn from benchmark suites that select for reproducible test-verified fixes — a population plausibly better documented than average, which may flatter the issue-text methods. The median repository here has 131 graph files; very large repositories are under-represented. [TODO: re-run the headline comparison split by repository size and report whether the reversal holds in the largest quartile]

Conclusion. Pull-request-level aggregation, Holm correction within comparison families and bootstrapping over repositories guard against the main inflation risks. Effect sizes accompany every test because several differences are significant and small.

7 CONCLUSION

We compared import-graph proximity and issue-text retrieval as reading orders for Python change tasks on 2,412 tasks from 1,873 pull requests across 113 repositories, under two evidence-based definitions of which files a change required. The comparison has no source-independent answer: the rankings the two sources induce over sixteen methods are statistically unrelated, with structural orderings winning on files the change references and lexical orderings winning on files the developer changed alongside it. The graph’s two-hop neighbourhood, the standard justification for structural retrieval, turns out to be half the repository. Fusing a structural and a lexical ordering is the only strategy that stays near the top under both sources, and it costs almost nothing. We recommend that context-retrieval work for code change report per evidence source, use a repository

prior rather than random order as its floor, and stop treating the dependency graph as the only signal in the room.

DATA AVAILABILITY

The replication package contains the import graphs at every base commit, the task definitions and evidence keys, the issue-token bags, the per-task orderings and coverage curves, and the analysis scripts that produce every number and figure in this paper. [TODO: archive and mint a DOI]

USE OF AI TOOLS

Claude (Anthropic) was used to write the analysis and method code, to run the experiments and to draft this manuscript. All numeric results are produced by scripts in the replication package and were copied mechanically from their output files; the author verified the design, the claims and the text. Bibliographic entries were drafted with AI assistance and [TODO: every reference must be checked against the published record before submission].

REFERENCES

- [1] Ibragim Badertdinov et al. 2025. SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents. arXiv preprint.
- [2] Shawn A. Bohner and Robert S. Arnold. 1996. *Software Change Impact Analysis*. IEEE Computer Society Press.
- [3] Zhaoling Chen et al. 2025. LocAgent: Graph-Guided LLM Agents for Code Localization. In *Annual Meeting of the Association for Computational Linguistics (ACL)*.
- [4] CoSIL authors. 2025. CoSIL: Issue Localization via LLM-Driven Iterative Code Graph Searching. arXiv preprint. TODO: confirm.
- [5] Bogdan Dit, Meghan Revelle, Malcom Gethers, and Denys Poshyvanyk. 2013. Feature Location in Source Code: A Taxonomy and Survey. *Journal of Software: Evolution and Process* 25, 1 (2013).
- [6] Paul Gauthier. 2023. Aider: Repository Map. <https://aider.chat/docs/repomap.html>. Accessed 2026-09-22.
- [7] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations (ICLR)*.
- [8] Andrew J. Ko, Brad A. Myers, Michael J. Coblenz, and Htet Htet Aung. 2006. An Exploratory Study of How Developers Seek, Relate, and Collect Relevant Information during Software Maintenance Tasks. *IEEE Transactions on Software Engineering* 32, 12 (2006).
- [9] Siru Ouyang et al. 2025. RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph. In *International Conference on Learning Representations (ICLR)*.
- [10] Stephen Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 3, 4 (2009).
- [11] Martin P. Robillard. 2008. Topology Analysis of Software Dependencies. *ACM Transactions on Software Engineering and Methodology* 17, 4 (2008).
- [12] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying LLM-based Software Engineering Agents. arXiv:2407.01489.
- [13] Annie T. T. Ying, Gail C. Murphy, Raymond Ng, and Mark C. Chu-Carroll. 2004. Predicting Source Code Changes by Mining Change History. *IEEE Transactions on Software Engineering* 30, 9 (2004).
- [14] Linghao Zhang et al. 2025. SWE-bench Goes Live! arXiv preprint.
- [15] Thomas Zimmermann, Peter Weißgerber, Stephan Diehl, and Andreas Zeller. 2005. Mining Version Histories to Guide Software Changes. *IEEE Transactions on Software Engineering* 31, 6 (2005).